What Does the Search Buy?
A Two-Axis Ablation of Alpha–Beta Agents for English Draughts

Anika Deshmukh[1], **Tomasz Wierzbicki**[2], **Carolina Méndez Ortega**[3], **Samuel Okonkwo**[1]
[1]Department of Computing Science, Riverbend Institute of Technology, Pune, India
[2]Faculty of Mathematics and Informatics, Vistula Polytechnic University, Kraków, Poland
[3]Departamento de Ciencias de la Computación, Universidad de Sierra Clara, Guadalajara, México
anika.deshmukh@rit-pune.ac.in, t.wierzbicki@mi.vistula-poly.edu.pl, carolina.mendez@cc.usierraclara.mx,
s.okonkwo@rit-pune.ac.in

Abstract

Game-playing agents are usually reported as a bundle: a paper states a final playing strength without saying which part of the machinery paid for it. We present a controlled attribution study for English draughts (American checkers) built on a single design decision — a one-ply baseline and a full iterative-deepening alpha–beta agent read the same static evaluation object, so a match between them measures the tree search and nothing else. Under that constraint the search scores 0.933 ± 0.062 per game against its own evaluation applied at one ply. We then ablate each classical enhancement along two axes that we argue should always be reported separately: a *node* ablation at fixed depth, which provably cannot change the move and can therefore be validated by an agreement check, and an *equal-time* ablation, in which saved nodes become extra depth and therefore won games. Alpha–beta with move ordering searches $18.2\times$ fewer nodes for a bit-identical root score, adding a transposition table reaches $25.7\times$, and the effective branching factor falls from 7.04 to 4.10 — about 55% of the way, in log terms, to the Knuth–Moore bound of $\sqrt{b} = 2.65$. On a 50 ms clock those savings buy roughly two extra plies and 0.656 ± 0.117 per game against bare alpha–beta. We also report three checkers-specific findings that follow from compulsory capture: counting legal moves is an actively harmful mobility proxy, quiescence needs no stand-pat option, and the single-reply extension is free to detect.

1 Introduction

English draughts is an unusually convenient laboratory for studying game-tree search. It is small enough to implement exactly — 32 playable squares, one piece type plus a promotion, and a movement rule that fits in a paragraph — yet far too large to enumerate. It is also historically central: it hosted the first substantial machine-learning program (Samuel 1959), one of the first world-champion-caliber programs (Schaeffer et al. 1992), and, in 2007, one of the largest game solutions ever computed (Schaeffer et al. 2007). That last fact gives work like ours a clean framing. We are not chasing novelty in the game; the answer is known, and it is a draw. We are measuring what it costs to approximate that answer, and which parts of a classical searcher pay for how much of the approximation.

The gap we address is one of attribution. Modern game AI has largely moved to learned evaluations and sampling-based planning, but the alpha–beta machinery underneath a great many systems is still the machinery of the 1970s and 1980s, and it is almost always reported as a bundle. A paper says that its agent searches to depth d with iterative deepening, a transposition table, killer moves and quiescence, and then reports a win rate. What it rarely says is how much of that win rate came from the evaluation rather than the search, or which enhancement bought which fraction of the depth. The one classic exception, Schaeffer (1989), measured tree sizes for combinations of enhancements but stopped at nodes: it never converted node savings into a clock and then into games, and it did not separate the ablations that can change the answer from the ones that cannot.

Separating those two kinds of ablation turns out to be the methodologically useful move, and it is the organising idea of this paper. Alpha–beta pruning, move ordering and (modulo one caveat we report in Section 6.4) a transposition table are *answer-preserving*: at a fixed depth they must return exactly the value that plain minimax returns, so they cannot change the move played and cannot, on their own, win a single game. Their entire value is in cost. Quiescence and search extensions are *answer-changing*: they exist precisely to return a different, better value. Reporting both in one table of win rates conflates a claim that can be checked for correctness with a claim that can only be measured statistically. We report them on separate axes and hold each to the standard appropriate to it.

We also found that the rule set matters more than we expected. English draughts makes capturing *compulsory*, and three of our design decisions are direct consequences of that rule rather than of anything carried over from chess. We report them because they are, as far as we can determine, not stated anywhere in the literature we surveyed.

Contributions.

- A three-layer separation of engine, agents and measurement harness in which the searching and non-searching agents share one evaluation object, so that a head-to-head match between them isolates the contribution of the tree search. Under this design the search is worth 0.933 ± 0.062 per game against its own evaluation at one ply.
- A two-axis ablation protocol — a fixed-depth node ablation that carries an agreement column as a live correct-

ness check, reported separately from an equal-time ablation in which node savings are converted into depth and then into game results. We argue this separation should be standard practice.

- A checkers-specific evaluation whose two most unusual terms are forced by compulsory capture: a mobility term that counts *slides only*, and an explicit capture-threat term. An earlier version of the agent that counted legal moves for mobility lost to the random baseline.
- A full set of measurements from one seeded, reproducible run: an $18.2\times$ node reduction at an identical answer, $25.7\times$ with the transposition table, an effective branching factor falling from 7.04 to 4.10, and roughly two extra plies at a fixed 50 ms clock.

2 Related Work

Checkers as an AI milestone. Samuel (1959) used checkers to introduce self-play weight tuning, and his evaluation — a weighted linear combination of hand-designed board features — is structurally what our own `Weights` object is. Chinook (Schaeffer et al. 1992) established what a world-class program in *our exact rule set* is made of: alpha–beta with the standard enhancement stack, a hand-tuned evaluation, endgame databases (Lake, Schaeffer, and Lu 1994) and an opening book. We deliberately omit the last two, so our agent is a study of the search core alone. The game was later solved (Schaeffer et al. 2005, 2007), which is why we frame our depth ladder as measuring the convergence of an approximation toward a known answer rather than as a search for new play.

Alpha–beta and its enhancements. Knuth and Moore (1975) analysed alpha–beta and established the $b^{\lceil d/2 \rceil}$ best case under perfect ordering, which we use in Section 6.4 as a yardstick rather than merely reporting speedups. Marsland (1986) surveys the pruning family. Our specific enhancements have canonical sources, cited at the point of description in Section 4.4: iterative deepening as a time-control mechanism (Slate and Atkin 1977), transposition tables and the hashing scheme they usually rest on (Zobrist 1970), the principal continuation and killer heuristics (Akl and Newborn 1977), and the history heuristic (Schaeffer 1989). We stop at plain alpha–beta with a full window; NegaScout/PVS (Pearl 1980; Reinefeld 1983) and MTD(f) (Plaat et al. 1996) are the next step we did not take.

Schaeffer (1989) is the closest prior work and the one a reader will ask about. It is our Section 6.4 experiment, run in 1989 by the author of Chinook, and it found that transposition tables and the history heuristic together account for close to all of the achievable tree-size reduction. We do not claim the *ranking* of enhancements as novel — it is not, and our numbers agree with it. Our deltas are methodological: that work measures nodes only, whereas we convert node savings into depth and then into game results under an equal-time budget; and it does not separate an ablation that cannot change the chosen move from one that can, which is the distinction we argue for. Campbell and Marsland (1983) is an earlier precedent for the general idea that tree size, not just strength, is the thing to measure.

Quiescence and the horizon effect. The horizon effect (Berliner 1973) and the quiescent/non-quiescent distinction that predates it (Shannon 1950) motivate searching past a nominal depth limit when the position is unsettled. Beal (1990) gives the formal treatment, built on a stand-pat value derived from the null move. Compulsory capture changes this in a way worth stating: our quiescence has *no stand-pat option*, because the side to move has no right to decline the capture. Anantharaman, Campbell, and Hsu (1990) is the reference for search extensions generally; singular extensions are expensive because identifying a singular move takes extra search, whereas under compulsory capture a single-reply node is free to detect — the move generator already reported exactly one move.

Learned and sampling-based alternatives. Blondie24 (Chellapilla and Fogel 1999) is the mirror image of this paper: it replaced the *evaluation* of a checkers minimax agent with an evolved network while keeping the search underneath, where we hold the evaluation fixed and vary the search. MCTS (Kocsis and Szepesvári 2006; Coulom 2006; Browne et al. 2012) and self-play reinforcement learning (Silver et al. 2018) are the modern alternatives. Two results explain why a well-characterised alpha–beta baseline remains useful in this game family. Ramanujan, Sabharwal, and Selman (2010) show that search spaces rich in shallow traps punish the averaging backups of UCT; checkers under compulsory capture is shallow-trap-rich almost by construction, since a single forced jump sequence can decide material immediately. And in international draughts, Li (2019) found that AlphaZero-style MCTS agents beat a random player but did not reliably beat an alpha–beta agent searching to depth 2 — suggestive rather than conclusive, since it is an unrefereed thesis with modest training compute, but directly on point.

3 English Draughts and What Compulsory Capture Changes

We implement the English draughts / American checkers rule set (Figure 1). Men slide one square diagonally forward; kings move one square in any diagonal direction and do *not* fly. A jump lands on the empty square immediately behind an adjacent enemy piece and must be continued while the same piece can jump again. Capturing is **compulsory**, though any available capture may be chosen — the longest is not forced. Crowning ends the turn even if further jumps would be available. A piece may not be jumped twice in one move, but captured pieces remain on the board until the move completes, so they can still block a landing square. A player with no legal move loses. Black moves first.

Three consequences of compulsory capture drive the rest of the paper.

(i) The branching factor is highly variable and often collapses to one. Whenever any jump exists, the legal-move list is the jump list, which is frequently a single move. This is what makes the single-reply extension nearly free: a node with one legal move costs one board application to look past, and the move generator has already identified it, so unlike singular extensions (Anantharaman, Campbell, and Hsu

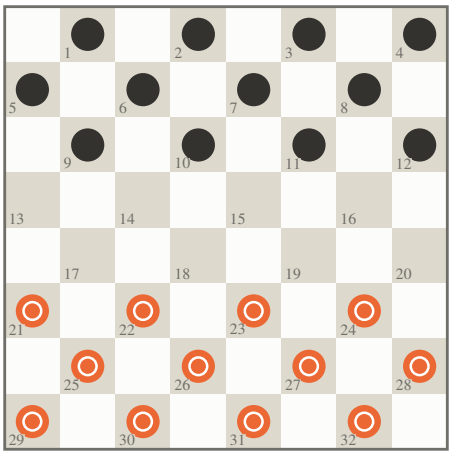


Figure 1: The starting position with the standard 1–32 square numbering. Play is on the 32 dark squares; Black (top, plain discs) moves first. Moves are written `11-15` for a slide and `11x18x25` for a jump chain.

1990) no extra search is needed to detect the case.

(ii) Legal-move count is a bad mobility proxy. Mobility has been an evaluation feature since Shannon (1950), and the obvious implementation — count the legal moves — is actively harmful here. A player who is about to win a piece has exactly one legal move, so a legal-move count scores being on the verge of winning material as being nearly stalemated. We found no prior statement of this in the literature we surveyed; our evaluation therefore counts *slides only* (Section 4.2).

(iii) A static score at a capture node is meaningless. The side to move has no say in what happens next, so evaluating the position statically evaluates a position that will never be reached in that form. This makes quiescence *necessary* in a stronger sense than in chess, and simultaneously *cheaper*, because the continuations are forced rather than merely available and each quiescence ply provably removes a piece.

Adjudication. To keep a batch of games finite we draw a game after 80 plies without a capture or man move (the traditional 40-move rule, counted in plies), on the third occurrence of a position, and at a hard cap of 400 plies. These are approximations of tournament practice, and they bias the results toward draws: positions that a stronger engine with endgame databases would convert are scored as half a point here. Serious 8×8 competition additionally uses the three-move ballot, a randomised opening restriction adopted precisely because the natural start position is drawish at high strength; we play from the natural start and discuss the consequences in Section 7.

4 Agents

4.1 Architecture and Reproducibility

The code base is three layers with a one-directional dependency. The engine (`checkers/`) knows nothing about agents: it accepts anything implementing a one-method `MoveChooser` protocol. The agents (`agents/`) depend on the engine. The measurement harness (`experiments/`) depends on both. This is not incidental tidiness — it is what makes the central experiment possible, because the harness can pair any two agents without either of them knowing what the other is.

`Board` is a frozen, hashable dataclass holding the 32 square contents plus the side to move, which is why positions can be used directly as transposition-table keys with no Zobrist scheme (Zobrist 1970). The trade-off is honest rather than favourable: Zobrist hashing exists to make position keys cheap to update incrementally and to give fixed-width, collision-tolerant keys, and we give up both in exchange for a value type the language hashes for us.

Every agent carries a seeded RNG derived from one base seed (20 260 824), and agent objects are reused across the games of an experiment so that stateful agents keep their state. The entire comparison therefore replays exactly. A suite of 76 unit tests covers the rules, the turn loop and the search; two of those tests assert that alpha–beta, move ordering and the transposition table return the same root score as the unenhanced search on every test position, which is the same property Section 6.4 measures at scale.

Term	Weight	Applies to
Man	100	each man
King	160	each king
Advancement	5	per row a man has advanced
Back row	8	a man still on its own back row
Edge	3	a piece on the a/h file
King centre	4	per step a king is toward the centre
Trade-down	6	material lead, scaled by pieces off
Mobility	2	per available *slide*
Capture threat	45	per piece in the longest jump

Table 1: The shared static evaluation, in hundredths of a man. The same `Weights` object is read by the one-ply `heuristic` agent and by the search agent at its leaves, which is what makes a match between them an isolation of search.

4.2 The Shared Static Evaluation

Table 1 gives the evaluation. It is a weighted linear combination of hand-designed features in the tradition of Shannon (1950) and Samuel (1959), returned from the side-to-move's point of view for negamax, with all positional terms folded into cached piece-square tables so that the leaf cost is one addition per occupied square. The feature set is not our contribution; holding it constant across agents is.

Two terms deserve the space, because both are forced by compulsory capture and neither appears in a standard chess-derived evaluation.

Mobility counts slides only. As argued in Section 3, counting legal moves inverts the sign of the feature exactly when it matters. We did not derive this from first principles — we found it. An earlier version of the agent whose mobility term counted legal moves lost matches to the *random*

Enhancement	Preserves the answer?
Alpha–beta pruning	yes
Iterative deepening	yes†
Transposition table	yes‡
PV / jump / promotion ordering	yes
Killer moves	yes
History heuristic	yes
Forced-capture quiescence	**no**
Single-reply extension	**no**

Table 2: The enhancement stack, and whether each one can change the move chosen at a fixed nominal depth. This column is the axis the paper is organised around; sources are cited at the point of description below. †: given no time limit. ‡: in theory; see Section 6.4 for a measured exception.

baseline, because it steered systematically toward positions in which it was about to lose material and away from positions in which it was about to win it. Splitting the move generator into `generate_captures` and `generate_slides`, and having the evaluation read only the latter, fixed it.

An explicit capture-threat term. A threatened capture is not yet material, but it is usually about to be, and without this term the static score of any position with a jump on the board is meaningless. The term adds 45 hundredths of a man per piece in the longest jump available *to the side to move only* — the opponent's threats may well evaporate once this move is played. This term is what lets the one-ply baseline be a respectable opponent rather than a piece-counter, and it is why quiescence, which addresses the same problem properly, buys less in games than one might expect (Section 6.6).

4.3 Baselines

Three non-searching agents. `random` plays uniformly at random over the legal moves. `greedy` takes the longest jump chain, breaking ties randomly, and otherwise moves at random; because capturing is compulsory it only ever gets to choose *which* jump chain to take. `heuristic` plays the move with the best static evaluation of the resulting position — one ply of lookahead, never considering the reply.

`heuristic` is the reference opponent for the depth ladder, and the choice matters. It is cheap enough (about 0.09 ms per move) to play hundreds of games against, it is strong enough that a one-ply blunder gets punished, and, critically, it shares the search agent's evaluation. A match between `minimax` and `heuristic` varies exactly one thing.

4.4 The Search Agent

The search agent is a negamax searcher with the stack in Table 2. We describe each briefly, with the property that matters for our design — whether it can change the chosen move at a fixed nominal depth.

Negamax with alpha–beta (Knuth and Moore 1975). One recursive routine scores every node from the side to move's point of view; the $[\alpha, \beta]$ window prunes branches that cannot affect the root decision. Answer-preserving by construction. At the root the window is opened one point below the current best so that ties remain distinguishable from fail-lows, which costs a few cutoffs but lets the agent break ties among equal-scoring root moves at random.

Iterative deepening (Slate and Atkin 1977). Depths $1, 2, 3, \ldots$ in turn; each pass seeds the transposition table and ordering tables for the next, which makes the deep pass cheaper than searching to that depth directly. It also lets an optional per-move time budget stop *between* passes without losing a move, which is what makes the equal-time experiment of Section 6.5 possible at all.

Transposition table. Positions recur constantly through transposed move orders. Entries store a depth and a bound flag (exact / lower / upper), so a shallow entry can narrow the window without being trusted outright, and mate scores are converted between "plies from the root" and "plies from here" on the way in and out. Replacement is depth-preferred.

Move ordering (Akl and Newborn 1977; Schaeffer 1989). Previous iteration's best move first, then jumps (longest first), then promotions, then the two killer moves for that ply, with the history score used only to break ties within a class — deliberately capped so a quiet move can never be promoted above a jump.

Forced-capture quiescence (Beal 1990). At depth zero, if the side to move has a jump available, the search keeps going rather than returning a static score. There is no stand-pat option, because there is no right to decline. Termination is immediate: every quiescence ply removes at least one piece. This is answer-changing by design, and a unit test pins the behaviour — on a two-for-one shot position (Black offers a man, Red's capture is forced, Black sweeps), the agent finds the combination at nominal depth 1 with quiescence on, needs depth 2 with it off, and the one-ply `heuristic` agent never finds it.

Single-reply extension (Anantharaman, Campbell, and Hsu 1990). A node with exactly one legal move does not consume depth, subject to a ply guard so a long forced chain cannot extend forever. Answer-changing, and, per Section 3, free to detect.

Mate scores are adjusted by ply, so the search prefers a fast win and a slow loss.

The design decision that enables this study is that *every one of these is switchable from the agent's specification string*, so that the ablations in Section 6 are configuration rather than code branches. Turning ordering and the table off gives a bare alpha–beta searcher; adding `alphabeta=false` gives textbook minimax. Nothing else about the agent differs between the rows of our tables.

5 Experimental Setup

The harness plays a batch of games per pairing, alternating colours every other game so that the first-move advantage cancels, and reuses agent objects across games. Games are scored 1 for a win, 0.5 for a draw and 0 for a loss, and we compare mean score per game.

The full comparison is a single seeded run (base seed $20\,260\,824$) taking about 1043 s of wall clock on one core of a Linux machine under Python 3.12, and it writes `results.json` and `results.md` from which every number in this paper is drawn. Uncertainties are ± 1.96 standard errors of the per-game score.

Agent	Games	W–L–D	Score	ms/move
Round robin among the baselines				
`heuristic`	400	399–0–1	0.999	0.20
`greedy`	400	108–286–6	0.278	<0.01
`random`	400	87–308–5	0.224	<0.01
`minimax:depth=6` against each baseline				
vs. `random`	30	30–0–0	1.000	123.8
vs. `greedy`	30	30–0–0	1.000	134.1
vs. `heuristic`	30	26–0–4	**0.933**	150.9

Table 3: Baseline ranking and the search agent against each baseline. The last row is the informative one: both sides use the same evaluation, so the gap is attributable to the tree search.

We flag the sample sizes up front, because they are the main limitation of this work. The baseline round robin plays 400 games per agent, but the searching pairings play only 16–30 games each: a depth-6 move costs about 150 ms, and a single 30-game match against the reference opponent takes 176 s. Where an interval straddles 0.5 we say so rather than reading a trend into it.

The two ablations use a fixed benchmark set of 24 positions sampled from seeded random playouts (every seventh ply from ply 6, discarding forced positions). Sampling from real games rather than from the opening keeps the node counts representative — the start position is unusually quiet and unusually easy to prune. Before every position, in every configuration, the engine's memoised move generators are cleared, so that no configuration inherits a warm cache from the one before it.

6 Results

6.1 Do the Baselines Rank as Expected?

Table 3 (top) gives the round robin. The ordering is the expected one, but two details are worth noting. First, the evaluation alone, at one ply, is very nearly unbeaten against non-evaluating play: 399 wins, 1 draw, 0 losses in 400 games. A one-ply agent with a reasonable feature set is already a serious opponent for anything that does not evaluate at all. Second, `greedy` at 0.278 is barely above `random` at 0.224. Taking the longest available jump is worth almost nothing on its own, which is a direct consequence of compulsory capture: the rule already forces the capture, so all `greedy` adds is a preference among jump chains.

6.2 Does Search Beat the Baselines?

Table 3 (bottom) gives the head-to-heads. `minimax` searching to depth 6 scores 1.000 against `random` and against `greedy`, winning every one of 60 games. Neither number is informative on its own; a one-ply agent already beats both.

The third number is the one this paper is built to produce. Against `heuristic` — its own evaluation, applied at one ply — the search scores $\mathbf{0.933 \pm 0.062}$, winning 26 of 30 and drawing 4, with no losses. Because the two agents read the same `Weights` object, that gap is attributable to the tree search and to nothing else: not to a better feature set, not to better-tuned weights, not to a different move generator. The cost is 150.9 ms per move against 0.09 ms, a factor of about 1700. Notably, all four draws came with the search playing Red; as Black it won all 15.

Configuration	Nodes	s	ebf	Speed-up	Agree
minimax	121,864	1.639	7.04	1.0×	100%
alpha–beta	13,417	0.164	4.88	9.1×	100%
+ ordering	6,701	0.093	4.34	18.2×	100%
+ TT	4,735	0.085	4.10	**25.7**×	96%
+ quiescence	5,746	0.089	4.23	21.2×	50%

Table 4: Node ablation: 24 sampled positions, each searched to depth 6, nodes and seconds per position. "ebf" is the effective branching factor $(\text{nodes})^{1/6}$; "Agree" is the fraction of positions on which the configuration returned exactly plain minimax's root score.

6.3 Does More Search Help?

Figure 2 gives the depth ladder both ways: against the fixed one-ply reference, and against the agent one ply shallower.

Against the reference the scores rise from 0.688 at depth 1 to 1.000 at depth 5, then fall to 0.917 at depth 6. Against the previous depth, the increments shrink monotonically and then invert: 0.719, 0.688, 0.688, 0.594, and finally 0.438 — depth 6 *losing* to depth 5. We report both findings rather than smoothing them.

The diminishing returns are the expected shape (Thompson 1982; Junghanns et al. 1997). Two mechanisms are at work and they are worth distinguishing. Against the fixed reference the effect is largely an artefact of saturation: once the agent scores 1.000 over 24 games there is no headroom left, and the ceiling, not the search, is what limits the number. This is precisely the masking effect Junghanns et al. (1997) describe — a weak opponent's own high error rate hides the improvement. Against the previous depth there is no ceiling, and the increments still shrink, from roughly 160 Elo-equivalent points per ply at depths 1–2 down to about 66 at depths 4–5.

The depth-5/6 inversion needs care. At 16 games the interval on that cell is 0.438 ± 0.198, which comfortably contains 0.5; the honest reading is that we cannot distinguish depth 6 from depth 5 with this sample, not that depth 6 is worse. Cost, on the other hand, is unambiguous: Figure 2(b) shows 0.21 ms per move at depth 1 rising to 153.8 ms at depth 6 — a factor of 3.0 per ply from depth 2 onward. Strength and cost should be read together, and past depth 4 this agent is buying very little for a great deal.

We also note an odd/even pattern consistent with the alternating-depth effect (Ferreira 2013): the two largest gains against the reference come at even depths 4 and, in the previous-depth series, depth 2. With 16–24 games per cell we can observe the pattern but not confirm it.

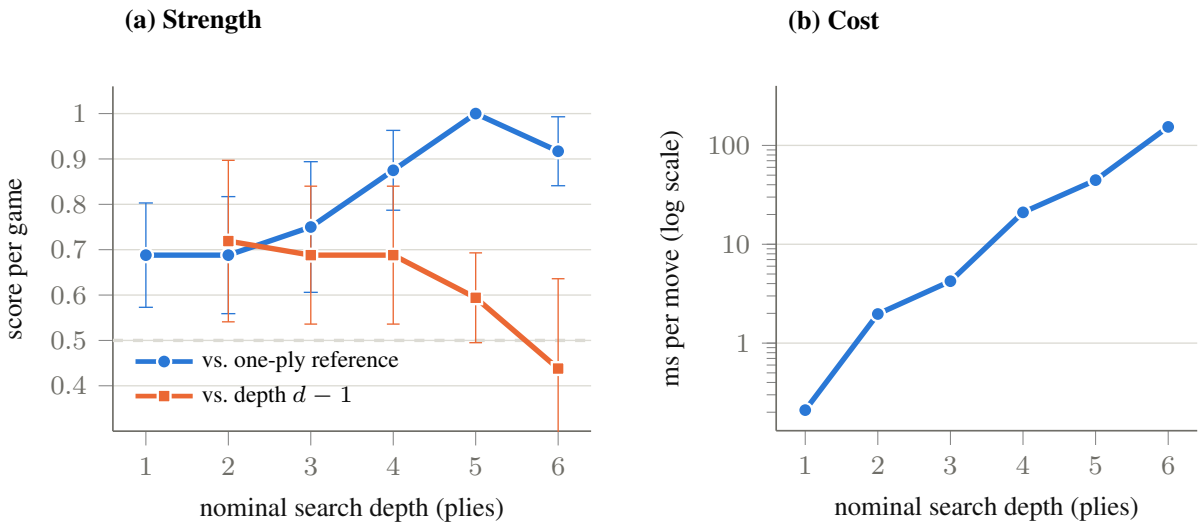


Figure 2: The depth ladder. (a) Score per game against the fixed one-ply reference opponent (blue circles, 24 games per point) and against the agent one ply shallower (orange squares, 16 games per point). Error bars are ± 1.96 standard errors; the dashed line is even. Saturation against the reference at depth 5 leaves no headroom, and the depth-6 point in each series should be read against its interval. (b) Cost of the same configurations, in milliseconds per move.

6.4 What Do the Techniques Buy? The Node Ablation

Table 4 and Figure 3(a) give the node ablation: the same 24 positions, the same depth 6, one configuration per row.

The headline numbers are that alpha–beta alone searches $9.1\times$ fewer nodes, move ordering takes that to $18.2\times$, and the transposition table to $25.7\times$. The effective branching factor falls from 7.04 to 4.10.

The *agreement* column is what we want to draw attention to, because it is not a performance number — it is a live correctness check, and it is available only because these ablations cannot change the answer. Alpha–beta and move ordering agree with plain minimax on 100% of positions, which is exactly what theory demands; anything less would be a bug, and this column would have found it. Quiescence's 50% is the *point* of quiescence rather than a defect: it is answer-changing by design, and on half the sampled positions it disagreed with the shallow static verdict.

Reading the effective branching factors against Knuth and Moore (1975) is more informative than reading the speed-ups. With $b \approx 7.04$ measured from plain minimax, perfect move ordering would give $\sqrt{b} \approx 2.65$. In log terms, bare alpha–beta captures 38% of that achievable reduction, our ordering scheme captures 50%, and the transposition table takes it to 55%. Slightly less than half of the theoretically available ordering gain is still on the table — which is a concrete, quantified argument for the aspiration windows and PVS we did not implement, and a more useful statement than "$25.7\times$ faster".

The transposition table's 96%. One row does not agree with plain minimax, and we flag it rather than bury it: on 1 of 24 positions the transposition table changed the root score. Our leading explanation is that a stored value in this implementation is not purely a function of (position, depth). Both the single-reply extension and the quiescence cut-off are gated on *absolute ply*, so the same position reached at a different ply can be searched differently under the same nominal depth, and depth-preferred replacement then permits one of those values to be reused in a context it was not computed for. This is the same transposition-table/search coupling that Plaat et al. (1996) analyse in the context of MTD(f). We have not isolated the disagreement to a specific position class, and we present the mechanism as a hypothesis. It is worth emphasising that we know about it *only* because the answer-preserving ablation carries an agreement column; a strength-only evaluation would have reported the transposition table as a clean win.

6.5 What Do They Buy on the Clock?

At a fixed depth the answer-preserving enhancements cannot win a game, so their value only appears once the search is given a clock instead of a depth. Giving each configuration 50 ms per move and letting iterative deepening go as deep as that buys yields Figure 3(b): the mean depth reached rises $4.17 \to 5.08 \to 5.38 \to 6.21$ as alpha–beta, ordering and the transposition table are added. The saved nodes are worth about **two extra plies**. The full agent reaches 6.17, slightly below the 6.21 of the same agent without quiescence, which is the expected price: quiescence spends nodes to improve the value rather than to save them.

In games at equal time (Table 5, bottom), the full agent scores 0.625 ± 0.141 against unpruned minimax and 0.656 ± 0.117 against bare alpha–beta. The second interval excludes 0.5; the first only just includes it.

The conclusion has to be drawn carefully. In nodes and

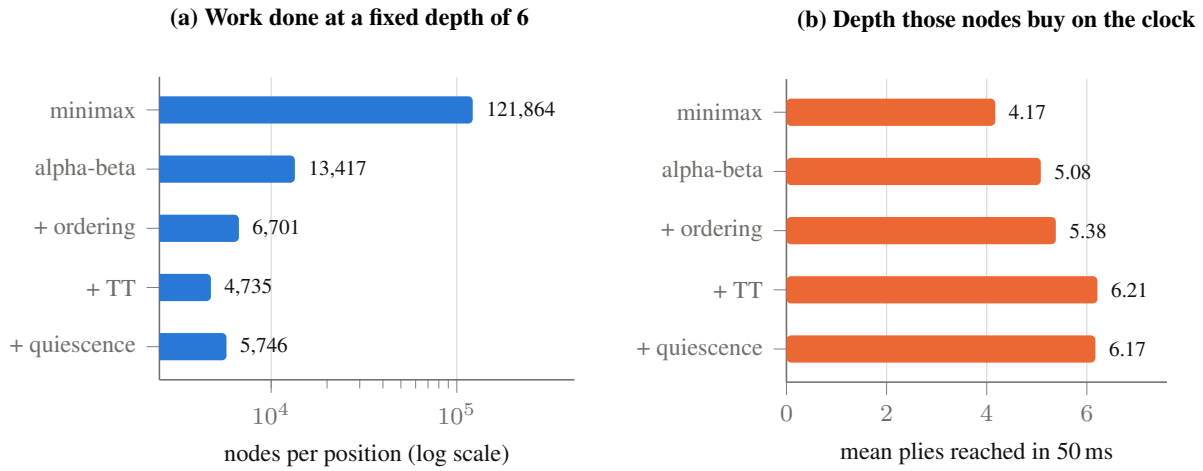


Figure 3: The two axes of the ablation. (a) Nodes searched per position at a fixed depth of 6 — a cost measurement, since these configurations return the same move. (b) Mean depth reached by the same configurations when each is given 50 ms per move instead of a depth. The node savings in (a) are what buy the extra plies in (b).

Match	W–L–D	Score	
Consecutive depths, 16 games each			
d2 vs. d1	9–2–5	0.719	± 0.178
d3 vs. d2	7–1–8	0.688	± 0.152
d4 vs. d3	7–1–8	0.688	± 0.152
d5 vs. d4	3–0–13	0.594	± 0.099
d6 vs. d5	4–6–6	0.438	± 0.198
Quiescence at equal depth, 30 games			
d4 vs. d4 no quiescence	9–7–14	0.533	± 0.132
Equal time (50 ms/move), 16 games each			
full vs. plain minimax	5–1–10	0.625	± 0.141
full vs. bare alpha–beta	5–0–11	0.656	± 0.117

Table 5: Head-to-head matches. Scores are for the first-named agent; uncertainties are ±1.96 standard errors of the per-game score.

in depth the enhancements are unambiguously worth it, by a factor of 20 and by two plies respectively. In *game score* at 50 ms per move on this board, the effect is real but modest, and the draw rate — 10 and 11 draws in 16 games — makes it hard to resolve at this sample size. Two plies at depth 6 in a game whose perfect-play value is a draw simply does not convert into wins at the rate the node counts might suggest. This is exactly the gap that a nodes-only report leaves unexamined, and it is the reason we think the second axis is worth the extra experiments.

6.6 Quiescence in Games

Quiescence is the one ablation that changes the move at a fixed depth, so it is also the one that has to be measured in games. At depth 4 over 30 games (Table 5, middle) the agent with quiescence scores 0.533 ± 0.132 against the same agent without it — indistinguishable from even, with 14 of 30 games drawn.

We think the explanation is the capture-threat term of Section 4.2, and that it is a genuine finding about evaluation design rather than a null result. That term is a cheap static approximation of what quiescence computes exactly; with it in place, the static evaluation of a position with a jump on the board is no longer meaningless, so the marginal value of searching the forced sequence properly is much smaller. The tactical unit test shows quiescence still does something the evaluation cannot — it finds a two-for-one shot two plies earlier — but at depth 4 in whole games, against an opponent with the same threat term, that advantage rarely decides a game.

7 Threats to Validity

Sample size is the main limitation. The searching pairings play 16–30 games. Intervals on those cells are ±0.10–0.20, wide enough that the depth-5/6 inversion, the quiescence match and the equal-time match against plain minimax are all individually unresolved. Heinz (2001) is the standard our depth-ladder methodology should be held to and we fall well short of it.

The evaluation is self-referential. The reference opponent shares the search agent's evaluation. That is the point — it is what isolates the search — but it means the two share blind spots, and a systematic error in the feature set is invisible to this comparison. The 0.933 figure measures what search adds *given this evaluation*, not what search adds in general.

Adjudication converts games into draws. Our threefold-repetition and no-progress rules end many long, materially-decided endgames as draws, and we have no endgame database (Lake, Schaeffer, and Lu 1994), so late-game strength is evaluation-limited in a way Chinook's was not. This compresses all our score rates toward 0.5, most visibly in the equal-time matches.

Timing mixes algorithmic and interpreter effects. All

measurements come from single-core Python on one machine. The node counts are implementation-independent; the milliseconds are not, and the depth reached in 50 ms would change with a faster move generator.

No external calibration. Every strength claim here is internal to this agent family. We have not played a public engine, so we cannot convert any of these score rates to an absolute scale.

8 Conclusion and Future Work

Holding the evaluation constant and varying only the search, a depth-6 alpha–beta agent scores 0.933 ± 0.062 per game against its own evaluation applied at one ply. The classical enhancements deliver an $18.2\times$ node reduction at a provably identical answer and $25.7\times$ with the transposition table, cut the effective branching factor from 7.04 to 4.10 — about 55% of the way in log terms to the Knuth–Moore bound — and convert into roughly two extra plies under a 50 ms clock and 0.656 ± 0.117 per game against bare alpha–beta.

The methodological point is worth carrying to other domains, and it is the thing we would most like a reader to take away. An ablation that *cannot* change the answer and one that *can* are different kinds of claim. The first can be validated for correctness — our agreement column caught a transposition-table anomaly that a strength-only comparison would have reported as a clean win — and it should be measured in nodes, then converted into depth under a clock, and only then into games. The second can only be measured in games, and needs the sample size to match. Bundling them into one table of win rates gives up a check that is free to run.

Compulsory capture turned out to shape the design more than the game's small size did. It makes legal-move count an actively harmful mobility proxy, removes the stand-pat option from quiescence, and makes single-reply extensions free to detect. The first of those cost us a version of the agent that lost to random play.

Future work. The cheapest high-value improvement needs no new experiments: fitting a Bradley–Terry or whole-history rating model (Coulom 2008) over all of our pairings at once would give every configuration a rating with a credible interval from the games already played, and would likely resolve the depth-5/6 inversion. Beyond that, the searching pairings should be rerun at 200+ games with a paired opening book drawn from the three-move ballot, using SPRT (Wald 1945) as computer chess does. On the search side, PVS (Reinefeld 1983) and MTD(f) (Plaat et al. 1996) would attack the 45% of the ordering gain we measured as still available, and an endgame database (Lake, Schaeffer, and Lu 1994) would remove the largest known weakness. On the evaluation side, the weights are hand-set, and TreeStrap-style bootstrapping from the search's own values (Veness et al. 2009; Baxter, Tridgell, and Weaver 2000) is the natural fit, since the agent already computes the training targets; rerunning the ablation afterward would show how much of the remaining gap is evaluation rather than search. Finally, the engine/agent/harness split is game-agnostic, so porting the same two-axis protocol to international draughts — flying kings, longest-capture rule, larger branching factor — would test whether the enhancement ranking is stable, and adding an MCTS agent (Browne et al. 2012) to the same harness against the same reference opponent would put the Ramanujan, Sabharwal, and Selman (2010) shallow-trap hypothesis to a direct test in a game where it should bite hardest.

References

Akl, S. G.; and Newborn, M. M. 1977. The Principal Continuation and the Killer Heuristic. In *Proceedings of the 1977 ACM Annual Conference*, 466–473.

Anantharaman, T.; Campbell, M. S.; and Hsu, F.-h. 1990. Singular Extensions: Adding Selectivity to Brute-Force Searching. *Artificial Intelligence*, 43(1): 99–109.

Baxter, J.; Tridgell, A.; and Weaver, L. 2000. Learning to Play Chess Using Temporal Differences. *Machine Learning*, 40(3): 243–263.

Beal, D. F. 1990. A Generalised Quiescence Search Algorithm. *Artificial Intelligence*, 43(1): 85–98.

Berliner, H. J. 1973. Some Necessary Conditions for a Master Chess Program. In *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI)*, 77–85.

Browne, C. B.; Powley, E.; Whitehouse, D.; Lucas, S. M.; Cowling, P. I.; Rohlfshagen, P.; Tavener, S.; Perez, D.; Samothrakis, S.; and Colton, S. 2012. A Survey of Monte Carlo Tree Search Methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1): 1–43.

Campbell, M. S.; and Marsland, T. A. 1983. A Comparison of Minimax Tree Search Algorithms. *Artificial Intelligence*, 20(4): 347–367.

Chellapilla, K.; and Fogel, D. B. 1999. Evolving Neural Networks to Play Checkers Without Relying on Expert Knowledge. *IEEE Transactions on Neural Networks*, 10(6): 1382–1391.

Coulom, R. 2006. Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search. In *Computers and Games (CG 2006)*, volume 4630 of *LNCS*, 72–83.

Coulom, R. 2008. Whole-History Rating: A Bayesian Rating System for Players of Time-Varying Strength. In *Computers and Games (CG 2008)*, volume 5131 of *LNCS*, 113–124.

Ferreira, D. R. 2013. The Impact of Search Depth on Chess Playing Strength. *ICGA Journal*, 36(2): 67–80.

Heinz, E. A. 2001. Self-Play, Deep Search and Diminishing Returns. *ICGA Journal*, 24(2): 75–79.

Junghanns, A.; Schaeffer, J.; Brockington, M.; Björnsson, Y.; and Marsland, T. 1997. Diminishing Returns for Additional Search in Chess. In *Advances in Computer Chess 8*, 53–67. Maastricht, The Netherlands: University of Maastricht.

Knuth, D. E.; and Moore, R. W. 1975. An Analysis of Alpha-Beta Pruning. *Artificial Intelligence*, 6(4): 293–326.

Kocsis, L.; and Szepesvári, C. 2006. Bandit Based Monte-Carlo Planning. In *Machine Learning: ECML 2006*, volume 4212 of *LNCS*, 282–293.

Lake, R.; Schaeffer, J.; and Lu, P. 1994. Solving Large Retrograde Analysis Problems Using a Network of Workstations. In *Advances in Computer Chess 7*, 135–162. Maastricht, The Netherlands: University of Limburg.

Li, M. 2019. *Applying Reinforcement Learning with Monte Carlo Tree Search to the Game of Draughts*. Master's thesis, University of Groningen.

Marsland, T. A. 1986. A Review of Game-Tree Pruning. *ICCA Journal*, 9(1): 3–19.

Pearl, J. 1980. Asymptotic Properties of Minimax Trees and Game-Searching Procedures. *Artificial Intelligence*, 14(2): 113–138.

Plaat, A.; Schaeffer, J.; Pijls, W.; and de Bruin, A. 1996. Best-First Fixed-Depth Minimax Algorithms. *Artificial Intelligence*, 87(1–2): 255–293.

Ramanujan, R.; Sabharwal, A.; and Selman, B. 2010. On Adversarial Search Spaces and Sampling-Based Planning. In *Proceedings of the 20th International Conference on Automated Planning and Scheduling (ICAPS)*, 242–245.

Reinefeld, A. 1983. An Improvement to the Scout Tree-Search Algorithm. *ICCA Journal*, 6(4): 4–14.

Samuel, A. L. 1959. Some Studies in Machine Learning Using the Game of Checkers. *IBM Journal of Research and Development*, 3(3): 210–229.

Schaeffer, J. 1989. The History Heuristic and Alpha-Beta Search Enhancements in Practice. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 11(11): 1203–1212.

Schaeffer, J.; Björnsson, Y.; Burch, N.; Kishimoto, A.; Müller, M.; Lake, R.; Lu, P.; and Sutphen, S. 2005. Solving Checkers. In *Proceedings of the 19th International Joint Conference on Artificial Intelligence (IJCAI)*, 292–297.

Schaeffer, J.; Burch, N.; Björnsson, Y.; Kishimoto, A.; Müller, M.; Lake, R.; Lu, P.; and Sutphen, S. 2007. Checkers Is Solved. *Science*, 317(5844): 1518–1522.

Schaeffer, J.; Culberson, J.; Treloar, N.; Knight, B.; Lu, P.; and Szafron, D. 1992. A World Championship Caliber Checkers Program. *Artificial Intelligence*, 53(2–3): 273–290.

Shannon, C. E. 1950. Programming a Computer for Playing Chess. *Philosophical Magazine*, 41(314): 256–275.

Silver, D.; Hubert, T.; Schrittwieser, J.; Antonoglou, I.; Lai, M.; Guez, A.; Lanctot, M.; Sifre, L.; Kumaran, D.; Graepel, T.; Lillicrap, T.; Simonyan, K.; and Hassabis, D. 2018. A General Reinforcement Learning Algorithm That Masters Chess, Shogi, and Go Through Self-Play. *Science*, 362(6419): 1140–1144.

Slate, D. J.; and Atkin, L. R. 1977. CHESS 4.5—The Northwestern University Chess Program. In Frey, P. W., ed., *Chess Skill in Man and Machine*, 82–118. Springer.

Thompson, K. 1982. Computer Chess Strength. In Clarke, M. R. B., ed., *Advances in Computer Chess 3*, 55–56. Pergamon Press.

Veness, J.; Silver, D.; Uther, W.; and Blair, A. 2009. Bootstrapping from Game Tree Search. In *Advances in Neural Information Processing Systems 22 (NIPS)*, 1937–1945.

Wald, A. 1945. Sequential Tests of Statistical Hypotheses. *Annals of Mathematical Statistics*, 16(2): 117–186.

Zobrist, A. L. 1970. A New Hashing Method with Application for Game Playing. Technical Report 88, Computer Sciences Department, University of Wisconsin–Madison. Reprinted in ICCA Journal 13(2):69–73, 1990.